Fahad Dashboard Site Build Process

Project overview

Fahad Dashboard is a React + Vite personal dashboard website with six connected sections:

1. Academic
2. My Plan
3. Movies
4. Job Prep
5. Books
6. University

The site is built as a single-page application. Navigation happens with React Router, and each section is a separate page component inside src/pages.

Main technologies used

- React 18
- Vite 5
- React Router DOM 6
- Browser localStorage for saving data
- CSS in src/styles.css
- Vercel-ready deployment setup

Folder structure

‘‘text
src/
App.jsx
main.jsx
styles.css
hooks/
useLocalStorage.js
pages/
Academic.jsx
Books.jsx
JobPrep.jsx
Movies.jsx
MyPlan.jsx
University.jsx

scripts/
generate-summary-pdf.mjs
docs/
Site-Build-Process-Summary.md
‘

How the site is built

1. App entry

The app starts from src/main.jsx. This file mounts the React app into the root HTML element and usually wraps the app with the router.

2. Main layout and routing

src/App.jsx is the central controller of the site.

It does three important jobs:

- defines the six dashboard pages
- creates the sidebar navigation
- connects routes to page components

The pages array in App.jsx is the backbone of the app. Each object includes:

- path
- label
- description
- accent
- element

That single array is reused for both:

- sidebar links
- route generation

This is a smart structure because one source controls navigation and page rendering together.

3. Shared visual design

The main visual system is in src/styles.css.

The stylesheet provides:

- app shell layout
- sidebar styling
- page panel styling
- hero section styling
- reusable card and table styles
- responsive behavior for tablet and mobile

The design uses:

- Fraunces for major headings
- Manrope for body text
- soft gradients
- glass-like panels
- rounded cards

4. Data persistence

The reusable hook src/hooks/useLocalStorage.js saves data inside the browser.

Pattern used:

1. read existing data from localStorage
2. fall back to a default value if nothing is stored
3. save updates automatically with useEffect
4. attach a version suffix like __v1 to storage keys

This means the whole dashboard works without a backend database.

Page-by-page build explanation

Academic page

src/pages/Academic.jsx is the largest and most advanced page.

It tracks:

- terms or semesters
- courses per term
- grades and credits

- automatic GPA and CGPA calculation
- class tests
- assignments and tasks

Important build ideas:

- helper functions convert grades to GPA points
- term GPA is calculated from completed courses
- final CGPA is calculated from completed terms
- tasks and tests are stored separately in local storage
- the page is broken into subcomponents like CGPASummary, TermBlock, and ClassTests

This page shows how to build a mini dashboard with calculations, forms, editing, filtering, and summary cards in one component.

My Plan page

src/pages/MyPlan.jsx is a personal goal tracker.

It supports:

- daily plans
- weekly plans
- monthly plans
- yearly plans
- status tracking
- category grouping
- analysis by time period

Important build ideas:

- plans are filtered by status and period
- analysis groups items by day, week, month, or year
- a custom SVG pie chart is generated directly in React
- success rate is calculated from completed and rejected items

This page is useful as a model for productivity and analytics features.

Movies page

src/pages/Movies.jsx tracks watched titles and a watchlist.

It includes:

- watched items
- type selection like movie, series, documentary, and anime
- rating input
- start and finish dates
- poster fetching from OMDb
- search suggestions from OMDb
- watchlist with posters
- year-by-year analysis

Important build ideas:

- fetch is used to talk to the OMDb API
- data from the API enriches the manual form data
- the page stores both a table view and card view of watched items
- a separate watchlist is maintained with its own local storage key

This page shows how the site mixes external APIs with local browser storage.

Job Prep page

src/pages/JobPrep.jsx is a career preparation dashboard.

It tracks:

- applications
- skills to learn
- learning resources
- multiple links per item
- requirements and deadlines
- recent activity

Important build ideas:

- status-based filtering for applications
- reusable link arrays for forms
- derived success-rate calculation
- normalization of older data shapes into the current format

This page is a good example of handling complex forms and structured lists.

Books page

src/pages/Books.jsx has two systems:

- reading list
- books to buy

It supports:

- title, author, rating, and dates
- finish tracking
- estimated buy cost
- book cover fetching from Open Library
- year-by-year reading analysis

Important build ideas:

- cover images are fetched when a book is added
- missing covers are hydrated later with useEffect
- analysis groups reading activity by year

This page shows async data enrichment and list analysis in a simple way.

University page

src/pages/University.jsx manages Masters and PhD targets.

It tracks:

- degree type
- university name
- ranking
- country
- application status
- requirements like IELTS, TOEFL, GRE, GMAT, SOP, CV, and LOR
- deadline
- tuition and scholarship
- notes and extra links

Important build ideas:

- tab-based switching between Masters and PhD

- expandable details for each university
- modal-style add/edit form
- search and status filters

This page is a good pattern for detailed record management.

Reusable patterns across the site

The same build method is repeated across almost all pages:

1. create local state with useState
2. store lists with useLocalStorage
3. create add, edit, delete, and reset functions
4. calculate summary values from stored data
5. render forms at the top
6. render cards, tables, or analysis sections below

This consistency makes the project easier to expand.

How to build a similar site step by step

Step 1. Create the project

‘bash
npm create vite@latest fahad-dashboard -- --template react
cd fahad-dashboard
npm install
npm install react-router-dom
‘

Step 2. Create the base files

Create:

- src/App.jsx
- src/styles.css
- src/hooks/useLocalStorage.js
- one page file for each dashboard section

Step 3. Add routing

Use react-router-dom and define all page routes in App.jsx.

Step 4. Build the layout

Create:

- sidebar
- page header
- content panel
- home page cards

Step 5. Build each page one by one

Recommended order:

1. Academic
2. My Plan
3. Books
4. Job Prep
5. University
6. Movies

This order works well because the first five can be built mostly with local forms and storage before adding movie API integration.

Step 6. Add local storage persistence

Use a reusable hook so every page can save data automatically without repeating the same logic.

Step 7. Add analytics and summaries

After CRUD is working, add:

- counters
- status summaries
- GPA calculations
- year analysis
- success rates
- charts

Step 8. Make the UI responsive

Use CSS media queries so the sidebar and content work on smaller screens.

Step 9. Build for production

‘bash
npm run build
‘

Vite outputs the final production files into dist/.

Step 10. Deploy

This site is ready for Vercel deployment.

Current deployment-friendly files:

- vercel.json
- vite.config.js

Strengths of this project structure

- simple and easy to understand
- no backend required
- each page is independent
- reusable storage hook
- centralized routing
- easy to deploy
- easy to add new dashboard sections

Possible future improvements

- split very large page files into smaller components
- move shared helpers into utility files
- hide API keys in environment variables
- add export and import of saved data
- add charts with a chart library if needed
- add authentication and cloud sync later

Commands to run the site

Development

‘bash
npm run dev
‘

Production build

‘bash
npm run build
‘

Preview build

‘bash
npm run preview
‘‘

Final summary

This site is built as a multi-page personal dashboard using React, Vite, React Router, custom CSS, and browser local storage.

The overall build flow is:

1. create a shared layout
2. define routes in one place
3. build one feature page at a time
4. save page data with local storage
5. add summaries, calculations, and analysis
6. deploy the final build on Vercel

If you want, this same structure can easily be extended with more pages like Finance, Habits, Health, or Notes.